Лабораторная работа №4

по дисциплине «Защита информации»

«Классический криптографический алгоритм DES»

1. Цель работы

Целью работы является знакомство с классическим криптографическим алгоритмом DES и его работой в режиме ECB.

2. Текст программы

Программа реализована на языке Go и не использует библиотечных реализаций блочного шифрования: все преобразования алгоритма DES выполнены непосредственно в коде. Программа откомпилирована и проверена; результаты ее выполнения приведены в пп. 3–5.

В состав программы входят следующие функциональные элементы.

Табличные константы. В программе заданы таблицы, определяющие все перестановки и нелинейные преобразования алгоритма: IP — начальная перестановка (таблица 1 методических указаний); IP-1 — обратная конечная перестановка (таблица 2); E — расширение 32-битного полублока до 48 бит (таблица 3); S1–S8 — таблицы нелинейных подстановок, восемь матриц размером 4x16 (таблица 4); P — перестановка внутри функции Фейстеля (таблица 5); PC-1 — перестановка-выбор ключа 64 -> 56 бит (таблица 6); LS — величины циклических сдвигов для 16 раундов (таблица 7); PC-2 — перестановка-выбор 56 -> 48 бит (таблица 8). Нумерация битов в таблицах ведется от 1 (старший бит) до 64 (младший).

Функции перестановок. Реализованы функции PermuteIP, PermuteIPInv, PermutePC1, PermutePC2, PermuteE и PermuteP. Каждая функция формирует выходной вектор в соответствии с таблицей: i-й бит входного вектора переносится в позицию, номер которой указан в i-м элементе таблицы. Перестановки PC-1 и PC-2 выполняют выбор битов ключа с отбрасыванием избыточных разрядов.

Формирование раундовых подключей. Функция GenerateKeys выполняет перестановку-выбор PC-1 исходного 64-битного ключа, разбиение полученного 56-битного вектора на два 28-битных полублока L и R, циклические сдвиги полублоков влево на LS(i) позиций в каждом из 16 раундов, объединение полублоков и перестановку-выбор PC-2. Результатом работы функции является массив из 16 48-битных раундовых подключей K1…K16.

Функция Фейстеля. Функция Feistel реализует раундовое преобразование f(R, K): расширение E 32-битного полублока до 48 бит; гаммирование полученного вектора с раундовым подключом (сложение по модулю 2); разбиение 48-битного вектора на восемь 6-битных векторов b1b2b3b4b5b6; выбор значения из i-го S-блока, при котором номер строки задается битами b1b6, а номер столбца — битами b2b3b4b5; объединение восьми 4-битных значений в 32-битный вектор; перестановку P.

Блочные функции шифрования и дешифрования. Функция EncryptBlock выполняет начальную перестановку IP, 16 раундов сети Фейстеля по правилам Li = Ri-1, Ri = Li-1 XOR f(Ri-1, Ki), перестановку полублоков (R16, L16) и обратную конечную перестановку IP-1. Функция DecryptBlock выполняет ту же последовательность действий, но раундовые подключи подаются в обратном порядке (K16…K1).

Режим ECB. Функции EncryptECB и DecryptECB обрабатывают входные данные порциями по 8 байт (64 бита): каждый блок интерпретируется как 64-битное число (порядок байтов big-endian) и преобразуется независимо от остальных блоков.

Дополнение по стандарту PKCS#7. Функция PKCS7Pad дополняет входные данные до кратности размеру блока; каждый байт дополнения равен количеству добавленных байтов. Функция PKCS7Unpad снимает дополнение и проверяет его корректность.

Управляющая функция main последовательно выполняет три эксперимента: базовое шифрование и дешифрование текста; искажение

одного бита шифрограммы и дешифрование искаженных данных; шифрование и повторное шифрование на слабом ключе.

Ниже приведен полный листинг программы (файл lab4/main.go).

```
package main

import (
    "bytes"
    "encoding/binary"
    "errors"
    "fmt"
)

//
-----------------------------------------------------------------------------------
-----------
// IP — таблица начальной перестановки (Initial Permutation) с нумерацией битов от 1
до 64
var IP = [64]int{
    58, 50, 42, 34, 26, 18, 10, 2,
    60, 52, 44, 36, 28, 20, 12, 4,
    62, 54, 46, 38, 30, 22, 14, 6,
    64, 56, 48, 40, 32, 24, 16, 8,
    57, 49, 41, 33, 25, 17, 9, 1,
    59, 51, 43, 35, 27, 19, 11, 3,
    61, 53, 45, 37, 29, 21, 13, 5,
    63, 55, 47, 39, 31, 23, 15, 7,
}

// IPInv — таблица 2 обратной конечной перестановки (IP-1)
var IPInv = [64]int{
    40, 8, 48, 16, 56, 24, 64, 32,
    39, 7, 47, 15, 55, 23, 63, 31,
    38, 6, 46, 14, 54, 22, 62, 30,
    37, 5, 45, 13, 53, 21, 61, 29,
    36, 4, 44, 12, 52, 20, 60, 28,
    35, 3, 43, 11, 51, 19, 59, 27,
    34, 2, 42, 10, 50, 18, 58, 26,
    33, 1, 41, 9, 49, 17, 57, 25,
}

//
-----------------------------------------------------------------------------------
-----------
// E — таблица 3 расширения (32 бита -> 48 бит)
var E = [48]int{
    32, 1, 2, 3, 4, 5,
    4, 5, 6, 7, 8, 9,
    8, 9, 10, 11, 12, 13,
    12, 13, 14, 15, 16, 17,
    16, 17, 18, 19, 20, 21,
    20, 21, 22, 23, 24, 25,
    24, 25, 26, 27, 28, 29,
    28, 29, 30, 31, 32, 1,
}

//
-----------------------------------------------------------------------------------
-----------
// SBox — таблицы 4 нелинейных подстановок (8 блоков по 4 строки и 16 столбцов)
var SBox = [8][4][16]byte{
    { // S1
        {14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7},
        {0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8},
        {4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0},
```

```
        {15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13},
    },
    { // S2
        {15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10},
        {3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5},
        {0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15},
        {13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9},
    },
    { // S3
        {10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8},
        {13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1},
        {13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7},
        {1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12},
    },
    { // S4
        {7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15},
        {13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9},
        {10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4},
        {3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14},
    },
    { // S5
        {2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9},
        {14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6},
        {4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14},
        {11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3},
    },
    { // S6
        {12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11},
        {10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8},
        {9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6},
        {4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13},
    },
    { // S7
        {4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1},
        {13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6},
        {1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2},
        {6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12},
    },
    { // S8
        {13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7},
        {1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2},
        {7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 13, 5, 8},
        {2, 1, 14, 7, 4, 10, 8, 13, 5, 12, 9, 0, 3, 5, 5, 11},
    },
}

//
-----------------------------------------------------------------------------------------
-----------
// P — таблица 5 перестановки внутри функции Фейстеля
var P = [32]int{
    16, 7, 20, 21, 29, 12, 28, 17, 1, 15, 23, 26, 5, 18, 31, 10,
    2, 8, 24, 14, 32, 27, 3, 9, 19, 13, 30, 6, 22, 11, 4, 25,
}

//
-----------------------------------------------------------------------------------------
-----------
// PC1 — таблица 6 перестановки-выбора PC-1 (56 элементов, нумерация битов от 1 до 64)
var PC1 = [56]int{
    57, 49, 41, 33, 25, 17, 9, 1, 58, 50, 42, 34, 26, 18,
    10, 2, 59, 51, 43, 35, 27, 19, 11, 3, 60, 52, 44, 36,
    63, 55, 47, 39, 31, 23, 15, 7, 62, 54, 46, 38, 30, 22,
    14, 6, 61, 53, 45, 37, 29, 21, 13, 5, 28, 20, 12, 4,
}

//
-----------------------------------------------------------------------------------------
-----------
```

```
// LS — таблица 7 циклических сдвигов для 16 раундов
var LS = [16]int{
    1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1,
}

//
-----------------------------------------------------------------------------------------
-----------
// PC2 — таблица 8 перестановки-выбора PC-2 (48 элементов)
var PC2 = [48]int{
    14, 17, 11, 24, 1, 5, 3, 28, 15, 6, 21, 10,
    23, 19, 12, 4, 26, 8, 16, 7, 27, 20, 13, 2,
    41, 52, 31, 37, 47, 55, 30, 40, 51, 45, 33, 48,
    44, 49, 39, 56, 34, 53, 46, 42, 50, 36, 29, 32,
}

//
-----------------------------------------------------------------------------------------
-----------
// PKCS7Pad дополняет срез байт до кратности blockSize (для DES это 8 байт / 64 бита)
func PKCS7Pad(data []byte, blockSize int) []byte {
    //
    padding := blockSize - (len(data) % blockSize)

    padBytes := bytes.Repeat([]byte{byte(padding)}, padding)
    return append(data, padBytes...)
}

//
-----------------------------------------------------------------------------------------
-----------
// PKCS7Unpad удаляет дополнение PKCS#7 после дешифрования
func PKCS7Unpad(data []byte) ([]byte, error) {
    length := len(data)
    if length == 0 || length%8 != 0 {
        return nil, errors.New("некорректная длина данных для снятия дополнения")
    }

    padLen := int(data[length-1])
    if padLen == 0 || padLen > 8 {
        return nil, errors.New("некорректное значение байта дополнения")
    }

    for i := length - padLen; i < length; i++ {
        if data[i] != byte(padLen) {
            return nil, errors.New("нарушена целостность байтов дополнения")
        }
    }

    return data[:length-padLen], nil
}

//
-----------------------------------------------------------------------------------------
-----------
// 64 -> 64
func PermuteIP(val uint64) uint64 {
    var res uint64

    for i := 0; i < 64; i++ {
        pos := IP[i]

        bit := (val >> (64 - pos)) & 1

        res |= bit << (63 - i)
```

```
    }
    return res
}

//
-----------------------------------------------------------------------------------------
-----------
func PermuteIPInv(val uint64) uint64 {
    var res uint64

    for i := 0; i < 64; i++ {
        pos := IPInv[i]
        bit := (val >> (64 - pos)) & 1
        res |= bit << (63 - i)
    }
    return res
}

//
-----------------------------------------------------------------------------------------
-----------
// 64 -> 56
func PermutePC1(key uint64) uint64 {
    var key56 uint64

    for i := 0; i < 56; i++ {
        // Номер бита в 64-битном ключе (от 1 до 64)
        pos := PC1[i]

        // Извлекаем бит из ключа (1-й бит — самый старший, сдвиг на 64 - pos)
        bit := (key >> (64 - pos)) & 1

        // Укладываем бит в 56-битное число от старшего к младшему (55 - i)
        key56 |= bit << (55 - i)
    }

    return key56
}

//
-----------------------------------------------------------------------------------------
-----------
// 56 -> 48
func PermutePC2(key uint64) uint64 {

    var key48 uint64
    for i := 0; i < 48; i++ {
        // Номер бита в 56-битном векторе (от 1 до 56)
        pos := PC2[i]

        // Извлекаем нужный бит. 1-й бит вектора cd — старший (находится на позиции
55)
        bit := (key >> (56 - pos)) & 1

        // Укладываем бит в 48-битный раундовый ключ от старшего к младшему (сдвиг 47
- i)
        key48 |= bit << (47 - i)
    }

    return key48
}
```

```
//
-------------------------------------------------------------------------------
-----------
// 32 -> 48
func PermuteE(val uint32) uint64 {
    var res uint64

    for i := 0; i < 48; i++ {
        pos := E[i]

        bit := (val >> (32 - pos)) & 1

        res |= uint64(bit) << (47 - i)
    }
    return res
}

//
-------------------------------------------------------------------------------
-----------
// 32 -> 32
func PermuteP(val uint32) uint32 {
    var res uint32

    for i := 0; i < 32; i++ {
        pos := P[i]

        bit := (val >> (32 - pos)) & 1

        res |= bit << (31 - i)
    }
    return res
}

//
-------------------------------------------------------------------------------
-----------
func GenerateKeys(srcKey uint64) [16]uint64 {
    var key56 uint64 = PermutePC1(srcKey)

    // L0 (в коде - левая часть) — это старшие 28 бит. Сдвигаем вправо на 28 позиций.
    var L uint32 = uint32(key56 >> 28)

    // R0 (в коде - правая часть) — это младшие 28 бит. Оставляем только их с помощью
маски.
    // 0x0FFFFFFF в двоичном виде — это ровно 28 единиц.
    var R uint32 = uint32(key56 & 0x0FFFFFFF)

    var keys [16]uint64

    for i := 0; i < 16; i++ {

        shift := LS[i]

        // Выполняем циклический сдвиг влево на shift позиций для каждой 28-битной
половины.
        // Конструкция (val >> (28 - shift)) переносит старшие выпадающие биты в
младшие разряды.
        L = ((L << shift) | (L >> (28 - shift))) & 0x0FFFFFFF
        R = ((R << shift) | (R >> (28 - shift))) & 0x0FFFFFFF

        // Склеиваем сдвинутые половины обратно в единый 56-битный вектор.
```

```
        // Левая половина сдвигается на 28 позиций влево, правая добавляется через
"ИЛИ".
        cd := (uint64(L) << 28) | uint64(R)

        // Применяем перестановку-выбор PC-2 для получения 48-битного ключа
        keys[i] = PermutePC2(cd)
    }

    return keys
}

//
-----------------------------------------------------------------------------------
-----------
// Feistel выполняет функцию f(R, K) для одного раунда
func Feistel(r uint32, key uint64) uint32 {

    // Расширение
    var e uint64 = PermuteE(r)

    // Гаммирование
    B := e ^ key

    // Преобразование в S-блоках (48 бит -> 32 бита)
    var s uint32
    for i := 0; i < 8; i++ {

        // 6-битный вектор
        vector6 := byte((B >> (42 - i*6)) & 0x3F)

        // 1,6
        k := ((vector6 >> 4) & 0x2) | (vector6 & 0x1)
        // 2,3,4,5
        l := (vector6 >> 1) & 0xF

        // Получаем 4-битное значение из S-блока
        val4 := uint32(SBox[i][k][l])

        // Укладываем 4 бита в итоговый 32-битный вектор
        s |= val4 << (28 - i*4)
    }

    // Перестановка P
    result := PermuteP(s)

    return result
}

//
-----------------------------------------------------------------------------------
-----------
// EncryptBlock шифрует один 64-битный блок
func EncryptBlock(block uint64, keys [16]uint64) uint64 {
    // Начальная перестановка
    block = PermuteIP(block)

    // Разделение на левую и правую 32-битные половины
    L := uint32(block >> 32)
    R := uint32(block & 0xFFFFFFFF)

    // 16 раундов сети Фейстеля
    for i := 0; i < 16; i++ {
```

```
        nextL := R
        nextR := L ^ Feistel(R, keys[i])

        L = nextL
        R = nextR
    }

    // Склейка половин в обратном порядке (R16, L16)
    reversedBlock := (uint64(R) << 32) | uint64(L)

    // Конечная обратная перестановка
    return PermuteIPInv(reversedBlock)
}

//
-----------------------------------------------------------------------------------------
-----------
// DecryptBlock расшифровывает один 64-битный блок
func DecryptBlock(block uint64, keys [16]uint64) uint64 {
    block = PermuteIP(block)

    L := uint32(block >> 32)
    R := uint32(block & 0xFFFFFFFF)

    // Те же 16 раундов, но ключи идут в обратном порядке (от 15 к 0)
    for i := 15; i >= 0; i-- {
        nextL := R
        nextR := L ^ Feistel(R, keys[i])

        L = nextL
        R = nextR
    }

    reversedBlock := (uint64(R) << 32) | uint64(L)

    return PermuteIPInv(reversedBlock)
}

//
-----------------------------------------------------------------------------------------
-----------
// EncryptECB шифрует срез байт (кратный 8) в режиме ECB
func EncryptECB(data []byte, keys [16]uint64) []byte {
    encrypted := make([]byte, len(data))
    for i := 0; i < len(data); i += 8 {
        block := binary.BigEndian.Uint64(data[i : i+8])
        encryptedBlock := EncryptBlock(block, keys)
        binary.BigEndian.PutUint64(encrypted[i:i+8], encryptedBlock)
    }
    return encrypted
}

//
-----------------------------------------------------------------------------------------
-----------
// DecryptECB расшифровывает срез байт (кратный 8) в режиме ECB
func DecryptECB(cipherData []byte, keys [16]uint64) []byte {
    decrypted := make([]byte, len(cipherData))
    for i := 0; i < len(cipherData); i += 8 {
        block := binary.BigEndian.Uint64(cipherData[i : i+8])
        decryptedBlock := DecryptBlock(block, keys)
        binary.BigEndian.PutUint64(decrypted[i:i+8], decryptedBlock)
    }
    return decrypted
}
```

```
//
-------------------------------------------------------------------------------
-----------
func main() {
    str := "Каморка его приходилась под самою кровлей высокого пятиэтажного дома и
походила более на шкаф, чем на квартиру."


    // Подготовка и дополнение исходных данных
    paddedData := PKCS7Pad([]byte(str), 8)


    // ============================================================================
    // ЭКСПЕРИМЕНТ 1: Обычное шифрование и дешифрование в режиме ECB
    fmt.Println("=== 1. ШИФРОВАНИЕ И ДЕШИФРОВАНИЕ (ECB) ===")
    fmt.Println("Исходная строка:", str)


    var srcKey uint64 = 0x13_34_57_79_9B_BC_DF_F1
    keys := GenerateKeys(srcKey)


    encryptedData := EncryptECB(paddedData, keys)
    fmt.Printf("Шифртекст (HEX): %X\n", encryptedData)


    decryptedData := DecryptECB(encryptedData, keys)
    unpaddedData, err := PKCS7Unpad(decryptedData)
    if err != nil {
        fmt.Println("Ошибка дешифрования/padding:", err)
        return
    }
    fmt.Println("Расшифрованная строка:", string(unpaddedData))
    fmt.Println()


    // ============================================================================
    // ЭКСПЕРИМЕНТ 2: Размножение ошибок (искажение 1 бита шифрограммы)
    fmt.Println("=== 2. ДЕМОНСТРАЦИЯ РАЗМНОЖЕНИЯ ОШИБОК ===")


    // Копируем шифртекст и инвертируем младший бит в первом байте (первый 64-битный
блок)
    corruptedCipher := make([]byte, len(encryptedData))
    copy(corruptedCipher, encryptedData)
    corruptedCipher[0] ^= 0x01 // Изменение ровно 1 бита


    fmt.Printf("Искаженный шифртекст (HEX): %X\n", corruptedCipher)


    corruptedDecrypted := DecryptECB(corruptedCipher, keys)
    // Так как искажен 1-й блок, дополнение в последнем блоке не пострадало
    corruptedUnpadded, err := PKCS7Unpad(corruptedDecrypted)
    if err != nil {
        fmt.Println("Ошибка padding:", err)
    } else {
        fmt.Printf("Дешифрованные байты 1-го блока (оригинал):   %X\n",
unpaddedData[:8])
        fmt.Printf("Дешифрованные байты 1-го блока (с ошибкой): %X\n",
corruptedUnpadded[:8])
        fmt.Println("Искаженный открытый текст:", string(corruptedUnpadded))
    }
    fmt.Println()


    // ============================================================================
    // ЭКСПЕРИМЕНТ 3: Работа алгоритма на слабых ключах (Таблица 9)
    fmt.Println("=== 3. РАБОТА НА СЛАБОМ КЛЮЧЕ ===")


    // Берем слабый ключ из Таблицы 9 методички
    var weakKey uint64 = 0x01_01_01_01_01_01_01_01
    weakSubkeys := GenerateKeys(weakKey)
```

```
    fmt.Printf("Слабый ключ: 0x%016X\n", weakKey)

    // Первое шифрование: C = DES(K, OT)
    weakEncryptedOnce := EncryptECB(paddedData, weakSubkeys)
    fmt.Printf("Шифртекст после 1-го шифрования (HEX): %X\n", weakEncryptedOnce)

    // Повторное ШИФРОВАНИЕ (EncryptECB, а не DecryptECB!): DES(K, DES(K, OT)) = OT
    weakEncryptedTwice := EncryptECB(weakEncryptedOnce, weakSubkeys)
    weakUnpadded, err := PKCS7Unpad(weakEncryptedTwice)
    if err != nil {
        fmt.Println("Ошибка padding:", err)
        return
    }
    fmt.Println("Текст после повторного шифрования тем же ключом:",
string(weakUnpadded))
}
```

3. Пример открытого текста и соответствующей ему шифрограммы

Базовый тест выполнен на 64-битном ключе K = 0x133457799BBCDFF1.

Открытый текст: «Каморка его приходилась под самою кровлей высокого пятиэтажного дома и походила более на шкаф, чем на квартиру.»

Открытый текст содержит 111 символов; в кодировке UTF-8 его длина составляет 204 байта. После дополнения по стандарту PKCS#7 длина входного сообщения составила 208 байт, то есть 26 блоков по 64 бита.

Полученная шифрограмма (HEX):

```
B9B0B980AFCE5141EE06260DE22AAB1314CC5068A5BD993BDF4414C0A62AB6294EFF3C88B209B
6BDEA54E09A3E499A06091361CCCF75AE0E8C67016110A49B745D96FBA2566B8970E5FE74DB77
9A600C3A50F7C00C4E7F36CBCCA6E830693971D8478391937585F98A4FA907CE632F002760FE2
783E25C45A7B04F30017EDF375D13ABEB42D434417C31F9D961AF3DFC067AD6DBF73AA8568B79
DF41419EE2AE3393DDCCA1E2AAB52F79E0A68376D441B7D787CAFBFA4C9E50A4EF7691ACB4E83
D22D8E1E71DCAA2475E089986CEC88E
```

Результат дешифрования шифрограммы совпал с исходным открытым текстом, что подтверждает корректность реализованных алгоритмов шифрования и дешифрования.

4. Шифрограмма с искаженным битом и соответствующий искаженный открытый текст

Для демонстрации свойства размножения ошибок в режиме ECB в шифрограмме, полученной в п. 3, был инвертирован ровно один бит —

младший бит первого байта шифрограммы. Значение первого байта изменилось с B9 (10111001 в двоичной записи) на B8 (10111000 в двоичной записи).

Искаженная шифрограмма (HEX):

```
B8B0B980AFCE5141EE06260DE22AAB1314CC5068A5BD993BDF4414C0A62AB6294EFF3C88B209B
6BDEA54E09A3E499A06091361CCCF75AE0E8C67016110A49B745D96FBA2566B8970E5FE74DB77
9A600C3A50F7C00C4E7F36CBCCA6E830693971D8478391937585F98A4FA907CE632F002760FE2
783E25C45A7B04F30017EDF375D13ABEB42D434417C31F9D961AF3DFC067AD6DBF73AA8568B79
DF41419EE2AE3393DDCCA1E2AAB52F79E0A68376D441B7D787CAFBFA4C9E50A4EF7691ACB4E83
D22D8E1E71DCAA2475E089986CEC88E
```

После дешифрования искаженной шифрограммы было выполнено сравнение первых восьми байт (первого 64-битного блока) открытого текста.

– оригинал: D09AD0B0D0BCD0BE — символы «Камо» в кодировке UTF-8, где каждая кириллическая буква занимает два байта;

– с ошибкой: 94E999B213799933.

Полученный искаженный открытый текст: «陲 у3рка его приходилась под самою кровлей высокого пятиэтажного дома и походила более на шкаф, чем на квартиру.» Отдельные байты искаженного блока не образуют корректных символов UTF-8 и отображаются служебными знаками.

Теоретическое пояснение. Изменение одного бита шифрограммы привело к полному искажению всего соответствующего 64-битного блока открытого текста: в данном эксперименте изменились все восемь байт блока. Причина состоит в свойствах раундовых преобразований алгоритма. Нелинейные подстановки S1–S8 не сохраняют линейные зависимости между входом и выходом, а перестановки E и P распределяют изменения отдельных бит по всему блоку, вследствие чего на каждом раунде сети Фейстеля искажение охватывает все новые биты (эффект размножения ошибок, или лавинный эффект). После 16 раундов единственный инвертированный бит криптограммы обусловливает искажение практически всех бит расшифрованного блока, которое не может быть устранено при обратном преобразовании. Вместе с тем все последующие блоки открытого текста («…рка его приходилась под самою кровлей высокого пятиэтажного дома и

походила более на шкаф, чем на квартиру.») были расшифрованы без искажений, поскольку в режиме ECB каждый 64-битный блок обрабатывается независимо от остальных: изменение одного блока шифрограммы не влияет на дешифрование соседних блоков. Корректность дополнения PKCS#7 также сохранилась, так как последний блок шифрограммы искажен не был.

5. Пример открытого текста и соответствующей шифрограммы, полученной при использовании слабого ключа

В соответствии с таблицей 9 методических указаний был выбран слабый ключ K = 0x0101010101010101, для которого L0 = [0]28 и R0 = [0]28. В качестве открытого текста использовался тот же текст, что и в п. 3.

Шифрограмма после первого шифрования DES(K, OT) (HEX):

```
6F3AAD982E3F4BE0E3CF9148F97324FA1300760381ABF87CB357647F468AEF7A1DE3508DFFE46
7664597AF3835352F7C612CAF3B412AB1C04147BBC2DF12CE2232CAF4F421E35C231F1C54AE15
52D8553EC0AE46F6EA1E77EE4F5B8A814A517477FDF0757748D87DC7615A312D9D3634F616A54
37F3358F85F18CA01687220FCFEAFD575362B522FFAEBBE09674635C72043ED2CB531EB444777
8E6B4E0E95F6B373BB8B5F7BCDCB5C9CC6222960F24DB242F5155FEA89602BC2C064DDD07E8E3
7C175A931811DDD401E05086D14084E
```

Результат повторного шифрования этой шифрограммы тем же ключом, то есть DES(K, DES(K, OT)): «Каморка его приходилась под самою кровлей высокого пятиэтажного дома и походила более на шкаф, чем на квартиру.» Полученный результат совпал с исходным открытым текстом.

Математическое объяснение эффекта. В ключе 0x0101010101010101 информационные биты всех восьми байтов равны нулю; единице равны только восьмые биты каждого байта, то есть биты контроля четности. При прохождении ключа через перестановку-выбор PC-1 контрольные биты в формируемый 56-битный рабочий ключ не переносятся, в результате чего оба 28-битных полублока L0 и R0 состоят из одних нулей: L0 = R0 = [0]28. При циклических сдвигах нулевых векторов во всех 16 раундах формируются 16 абсолютно идентичных раундовых подключей: K1 = K2 = … = K16. Так как алгоритм дешифрования отличается от алгоритма

шифрования только обратным порядком подачи подключей, при одинаковых подключах прямой и обратный порядки совпадают, и операция прямого шифрования на таком ключе становится инволюцией, то есть полностью совпадает с дешифрованием: DES(K, DES(K, OT)) = OT. Данное свойство слабых ключей является одной из известных уязвимостей алгоритма DES.

6. Выводы по работе

1. Изучен принцип работы классического криптографического алгоритма DES, построенного на 16-раундовой сети Фейстеля. Алгоритм сочетает перестановки (начальную IP, обратную IP-1, расширение E, перестановку P, перестановки-выборы PC-1 и PC-2), гаммирование 48-битным раундовым подключом и нелинейные подстановки в блоках S1–S8; криптостойкость DES определяется главным образом нелинейностью S-блоков.

2. Реализована программная модель DES в базовом режиме ECB, обеспечивающая шифрование и дешифрование текстовых данных произвольной длины с дополнением блоков по стандарту PKCS#7. В режиме ECB каждый 64-битный блок шифруется одним и тем же ключом независимо от остальных блоков; для шифрования используются 56 бит ключа из 64, оставшиеся 8 бит предназначены для контроля четности.

3. Экспериментально подтверждено свойство размножения ошибок в режиме ECB: инвертирование одного бита криптограммы привело к полному искажению соответствующего 64-битного блока открытого текста (все восемь байт блока изменились), тогда как все последующие блоки были расшифрованы без искажений. Таким образом, ошибка локализуется в пределах одного блока и не распространяется на соседние блоки.

4. Выявлены недостатки режима ECB: одинаковым блокам открытого текста при одном и том же ключе соответствуют одинаковые блоки шифрограммы, что дает криптоаналитику дополнительные сведения о структуре сообщения; кроме того, режим не обеспечивает имитостойкости —

повторное использование перехваченной криптограммы позволяет имитировать ответное сообщение.

5. Отмечены уязвимости самого алгоритма DES: малая эффективная длина ключа (56 бит) делает принципиально возможной атаку полным перебором ключевого пространства, а наличие не менее четырех слабых ключей (в частности, 0x0101010101010101) превращает шифрование в самообратную операцию: DES(K, DES(K, OT)) = OT. Совокупность перечисленных недостатков ограничивает практическую применимость алгоритма DES в современных системах защиты информации.